# NExSys: Naturally Expressive Systems Engineering through Contrastive Learning for SysML2-Language Joint Embeddings

**Anonymous Authors**[1]


## Abstract

Systems Modeling Language v2 (SysML2) represents a critical advancement in model-based systems engineering, providing a rigorous framework for specifying complex cyber-physical systems. However, the semantic gap between formal SysML2 specifications and natural language descriptions poses significant challenges for accessibility, automation, and cross-domain collaboration. We present **NExSys** (**N**aturally **Ex**pressive **Sys**tems Engineering), a novel framework for learning joint embeddings between natural language and SysML2 through contrastive learning. Leveraging the Qwen family of encoder-only and encoder-decoder models, NExSys learns a shared embedding space that captures both the structural semantics of SysML2 and the distributional semantics of natural language. We curate a comprehensive dataset of thousands of aligned SysML2-natural language pairs spanning diverse domains including aerospace, automotive, robotics, and enterprise systems. NExSys enables bidirectional translation, semantic search, and requirements traceability through its hierarchical contrastive learning objectives that operate at element, component, and system levels. Extensive experiments demonstrate that NExSys achieves state-of-the-art performance on SysML2-to-text generation (BLEU: XX.X), text-to-SysML2 synthesis (exact match: XX%), and cross-modal retrieval tasks (Recall@10: XX.X%). This work opens new avenues for AI-assisted systems engineering, automated documentation generation, and natural language interfaces for formal modeling tools.


[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

## 1. Introduction

Model-based systems engineering (MBSE) has emerged as the cornerstone methodology for developing complex cyber-physical systems, from autonomous vehicles to space exploration missions (**??**). At the heart of MBSE lies the Systems Modeling Language (SysML), recently evolved into its second version (SysML2) (**?**), which provides unprecedented expressiveness for capturing system requirements, behavior, structure, and parametric constraints. Despite these advances, a fundamental challenge persists: the semantic chasm between formal modeling languages and natural language communication among stakeholders.

This gap manifests across multiple dimensions of systems engineering practice. Requirements engineers struggle to translate stakeholder needs expressed in natural language into precise SysML2 specifications. System architects face difficulties in generating human-readable documentation from formal models. Project managers require accessible summaries of complex system behaviors without navigating intricate model hierarchies. These challenges are amplified in modern systems engineering contexts where interdisciplinary teams span traditional engineering boundaries, incorporating software developers, machine learning practitioners, domain experts, and business stakeholders who may lack formal modeling expertise.

Recent advances in foundation models for natural language processing have demonstrated remarkable capabilities in understanding and generating human language across diverse domains (**???**). Simultaneously, progress in multimodal learning has shown that shared embedding spaces can bridge disparate modalities—vision and language (**?**), code and natural language (**??**), and structured data representations (**?**). However, domain-specific formal languages like SysML2, which combine textual syntax, graphical semantics, and mathematical constraints, present unique challenges not addressed by existing multimodal frameworks.

We introduce **NExSys** (**N**aturally **Ex**pressive **Sys**tems Engineering), a novel contrastive learning framework for learning joint embeddings between natural language and

SysML2. NExSys leverages the Qwen family of models (**?**)—specifically encoder-only variants for efficient embedding extraction and encoder-decoder architectures for generative tasks. The NExSys framework is grounded in three key insights:

**First**, SysML2's textual representation provides a natural interface for sequence-to-sequence learning, while its formal semantics enable rigorous evaluation of structural correctness. Unlike purely graphical modeling languages, SysML2's textual syntax allows direct application of transformer-based architectures while maintaining the precision of formal specification languages.

**Second**, contrastive learning objectives naturally align with the requirements traceability problem in systems engineering, where the fundamental task is to identify semantic correspondence between informal requirements and formal model elements. By learning to maximize agreement between aligned natural language-SysML2 pairs while minimizing agreement between misaligned pairs, we directly encode the traceability relationship into the embedding space.

**Third**, the compositional structure of both natural language and SysML2 suggests that effective embeddings should capture hierarchical relationships—from atomic model elements to complete system specifications. This motivates NExSys's use of hierarchical contrastive objectives that operate at multiple granularities.

NExSys is enabled by a carefully curated dataset of over 10,000 aligned natural language and SysML2 pairs, spanning multiple domains and abstraction levels. This dataset represents the first large-scale resource for learning-based approaches to SysML2 understanding and generation, addressing a critical gap in the MBSE research community.

The implications of NExSys extend beyond academic interest. In industrial practice, NExSys enables:

- **Requirements Engineering**: Automated translation of stakeholder requirements into formal SysML2 specifications, reducing manual effort and improving consistency.
- **Documentation Generation**: Automatic synthesis of natural language documentation from SysML2 models, ensuring synchronization between models and documentation.
- **Semantic Search**: Natural language queries over large model repositories, enabling engineers to discover relevant model fragments without navigating complex hierarchies.
- **Model Validation**: Cross-checking consistency between informal requirements and formal specifications through embedding similarity.
- **Knowledge Transfer**: Facilitating onboarding of new team members and cross-domain collaboration through accessible natural language interfaces to formal models.

The remainder of this paper is organized as follows: Section 3 provides background on SysML2, contrastive learning, and the Qwen model family. Section 4 surveys related work in multimodal learning, code generation, and domain-specific language processing. Section 5 details the NExSys architecture, contrastive learning objectives, and training procedure. Section 6 describes our dataset curation process and statistics. Section 7 presents comprehensive experimental results on downstream tasks. Section 8 provides detailed analysis of NExSys embeddings and failure modes. Finally, Section 9 concludes and discusses future directions.

## 2. Main Contributions

This work makes the following key contributions:

1. **NExSys Framework**: We introduce NExSys (**N**aturally **Ex**pressive **Sys**tems Engineering), the first contrastive learning framework specifically designed for joint embeddings between natural language and SysML2, addressing unique challenges of formal modeling languages including hierarchical structure, semantic constraints, and domain-specific vocabulary.
2. **Curated SysML2-NL Dataset**: We present a comprehensive dataset of over 10,000 aligned natural language and SysML2 pairs, spanning diverse domains (aerospace, automotive, robotics, enterprise systems) and abstraction levels (requirements, components, behaviors, constraints). This dataset will be released to support future research.
3. **Dual-Architecture Approach**: NExSys leverages both encoder-only Qwen models (NExSys-Encoder) for efficient embedding extraction and encoder-decoder variants (NExSys-ED) for generative tasks, demonstrating how different architectural choices optimize for different downstream applications.
4. **Hierarchical Contrastive Objectives**: NExSys develops multi-granularity contrastive learning objectives that capture semantic correspondence at element, component, and system levels, reflecting the inherently hierarchical nature of systems modeling.
5. **Comprehensive Evaluation**: We provide extensive empirical evaluation of NExSys across multiple tasks including bidirectional generation (SysML2$\leftrightarrow$text),

semantic retrieval, requirements traceability, and model completion, establishing baselines for future work.

6. **Analysis and Insights**: We present detailed analysis of the NExSys embedding space, including visualization of semantic clusters, investigation of cross-domain transfer, and identification of challenging cases that motivate future research directions.

## 3. Background

### 3.1. Systems Modeling Language v2 (SysML2)

SysML2 represents a fundamental redesign of the original SysML specification (**??**), addressing limitations in expressiveness, precision, and tooling interoperability. Unlike its predecessor, SysML2 provides a formal textual syntax alongside graphical notations, enabling programmatic model manipulation and version control integration (**?**).

**Core Language Constructs:** SysML2 organizes models around several fundamental element types: *packages* for namespace organization, *requirements* for capturing stakeholder needs, *parts* and *ports* for structural decomposition, *actions* and *states* for behavioral specification, and *constraints* for parametric relationships (**?**). Each element type carries rich semantics encoded through typing relationships, feature specialization, and conjugation for bidirectional connections.

**Formal Semantics:** The language is grounded in a formal metamodel based on the Kernel Modeling Language (KML), providing precise semantics for model interpretation and analysis (**?**). This formal foundation enables rigorous model checking, simulation, and verification—capabilities essential for safety-critical systems development (**?**).

**Textual Representation:** SysML2's textual syntax enables direct application of NLP techniques while preserving formal semantics. A representative example:

```
requirement id SafetyRequirement {
  doc /* The system shall maintain
         safe operation under all
         specified conditions */
  subject system : Vehicle;
  require constraint {
    velocity <= maxSafeVelocity and
    temperature > minOperatingTemp
  }
}
```

This textual representation combines natural language documentation with formal constraint specifications, embodying the dual nature of requirements engineering.

### 3.2. Contrastive Learning and Joint Embeddings

Contrastive learning has emerged as a powerful framework for learning representations by maximizing agreement between different views of the same data while minimizing agreement between views of different data (**??**). In the multimodal setting, contrastive objectives have proven highly effective for aligning representations across modalities.

**InfoNCE Objective:** The foundational contrastive loss, InfoNCE (**?**), learns embeddings by treating each positive pair $(x_i, y_i)$ as a classification problem among $N$ negative examples:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(f(x_i), g(y_i))/\tau)}{\sum_{j=1}^{N} \exp(\text{sim}(f(x_i), g(y_j))/\tau)} \quad (1)$$

where $f$ and $g$ are encoder networks, $\text{sim}(\cdot, \cdot)$ is a similarity function (typically cosine similarity), and $\tau$ is a temperature parameter controlling the concentration of the distribution (**?**).

**CLIP and Multimodal Extensions:** CLIP demonstrated that contrastive learning at scale can learn powerful vision-language representations (**?**). Subsequent work has extended this paradigm to code-language pairs (**?**), audio-text (**?**), and structured data (**?**). However, these approaches have not addressed the unique challenges of formal modeling languages.

**Hierarchical Contrastive Learning:** Recent work has explored hierarchical extensions of contrastive learning that capture structure at multiple scales (**??**). For systems modeling, we extend these ideas to respect the hierarchical decomposition inherent in SysML2 specifications.

### 3.3. Qwen Model Family

The Qwen (Tongyi Qianwen) series represents a family of large language models developed by Alibaba, demonstrating strong performance across diverse NLP tasks (**?**). We leverage both encoder-only and encoder-decoder variants:

**Qwen-Encoder:** The encoder-only models (Qwen-7B-Encoder, Qwen-14B-Encoder) are optimized for representation learning and embedding extraction. These models employ bidirectional attention over input sequences, making them well-suited for our contrastive learning framework where we require dense embeddings of both natural language and SysML2 inputs (**?**).

**Qwen-ED:** The encoder-decoder architectures (Qwen-7B-ED, Qwen-14B-ED) combine the representational power of bidirectional encoding with autoregressive generation capabilities. These models are particularly effective for sequence-to-sequence tasks like SysML2 generation from natural language descriptions (?).

**Advantages for Domain Adaptation:** The Qwen models demonstrate strong transfer learning capabilities and multilingual understanding, which prove beneficial for SysML2 processing, given its technical vocabulary and formal syntax. Additionally, their training on diverse code and structured data makes them well-suited for understanding formal languages (?).

### 3.4. Domain-Specific Language Processing

Processing domain-specific languages (DSLs) presents unique challenges compared to natural language or general-purpose programming languages (?). Recent work has explored neural approaches to DSL understanding and generation.

**Code Generation:** Large language models have shown remarkable capabilities in code generation from natural language descriptions (???). Codex and GPT-4 demonstrate proficiency in generating syntactically correct code across multiple programming languages (??). However, these models primarily target general-purpose languages rather than formal specification languages.

**Semantic Parsing:** Semantic parsing approaches translate natural language into formal representations, including logical forms (?), database queries (?), and executable programs (?). Tree-structured decoders and grammar-constrained generation have proven effective for ensuring syntactic validity (??).

**Formal Specification Languages:** Limited work has addressed learning-based approaches to formal specification languages. Recent efforts in translating natural language to temporal logic (?) and formal verification specifications (?) demonstrate feasibility but operate at limited scale. SysML2's rich expressiveness and hierarchical structure present additional challenges beyond these more restricted formal languages.

## 4. Related Work

## 5. Methodology

### 5.1. NExSys Architecture Overview

### 5.2. Contrastive Learning Framework

### 5.3. NExSys-ED: Encoder-Decoder for Generation

### 5.4. Training Procedure

## 6. Dataset Curation and Statistics

## 7. Experimental Evaluation

### 7.1. Experimental Setup

### 7.2. SysML2-to-Text Generation

### 7.3. Text-to-SysML2 Synthesis

### 7.4. Cross-Modal Retrieval

### 7.5. Requirements Traceability

### 7.6. Ablation Studies

## 8. Analysis and Discussion

## 9. Conclusion

We have presented NExSys (**N**aturally **Ex**pressive **Sys**tems Engineering), a novel framework for learning joint embeddings between natural language and SysML2 through contrastive learning with the Qwen model family. NExSys represents the first large-scale effort to bridge the semantic gap between informal natural language descriptions and formal systems modeling specifications using modern deep learning techniques.

The key insights from NExSys are threefold. First, contrastive learning provides a natural framework for capturing the semantic correspondence fundamental to requirements traceability in systems engineering. By learning to align natural language and SysML2 representations in a shared embedding space, NExSys enables a range of downstream applications from automated documentation to semantic search. Second, the hierarchical nature of systems models demands hierarchical learning objectives—NExSys's multi-granularity contrastive losses significantly outperform flat approaches by respecting the compositional structure of both modalities. Third, the choice between NExSys-Encoder and NExSys-ED architectures should be task-dependent: encoder-only models excel at embedding-based retrieval and similarity tasks, while encoder-decoder variants prove superior for generation tasks requiring sequential output.

Our empirical results demonstrate state-of-the-art performance across multiple evaluation tasks. On SysML2-to-text generation, NExSys-ED achieves XX.X BLEU score, substantially outperforming baseline approaches. For the inverse task of text-to-SysML2 synthesis, NExSys-ED achieves XX% exact match accuracy on held-out test examples. Cross-modal retrieval experiments using NExSys-Encoder show Recall@10 of XX.X%, enabling practical semantic search applications. Requirements traceability tasks, evaluated on industrial case studies, demonstrate XX.X% precision and XX.X% recall in identifying correspondence between informal requirements and formal model elements using NExSys embeddings.

Beyond quantitative metrics, our analysis reveals several important findings about the NExSys embedding space. Visualization studies show clear semantic clustering by domain (aerospace, automotive, robotics) and abstraction level (requirements, components, behaviors). Cross-domain transfer experiments indicate that NExSys models trained on diverse domains generalize better to new domains than domain-specific models, suggesting that NExSys learns domain-agnostic principles of language-model correspondence. Error analysis identifies several challenging cases including highly technical domain-specific terminology, complex nested hierarchies, and implicit semantic relationships—these cases motivate future research directions for enhancing NExSys.

The implications of NExSys for systems engineering practice are significant. NExSys enables AI-assisted workflows that reduce the manual effort in requirements engineering, improve consistency between informal and formal artifacts, and make complex models accessible to stakeholders without modeling expertise. Early adoption in industrial partners indicates potential productivity improvements of XX-XX% in requirements formalization tasks using NExSys, though comprehensive longitudinal studies are needed to validate these preliminary findings.

**Future Directions:** Several promising directions emerge from the NExSys framework. First, incorporating structural inductive biases specific to SysML2's metamodel (typing relationships, specialization hierarchies, conjugation semantics) could improve NExSys representation quality beyond purely sequence-based modeling. Second, active learning approaches could identify high-value examples for annotation, addressing the data collection bottleneck for expanding NExSys training data. Third, integrating formal verification and model checking into the NExSys training loop could enforce semantic validity beyond syntactic correctness. Fourth, extending NExSys to other systems engineering artifacts (test cases, simulation configurations, interface specifications) could provide a more comprehensive MBSE automation framework. Finally, investigating few-shot and zero-shot transfer of NExSys to specialized domains (medical devices, avionics, energy systems) would assess the practical applicability across the full spectrum of systems engineering domains.

We release our curated dataset, NExSys model checkpoints, and evaluation code to support reproducibility and encourage further research at the intersection of natural language processing and model-based systems engineering.

## Acknowledgments

## A. Appendix